Vikram Penumarti (920928592)

Ayaan Puri (920893614)

Part 1a.

File locations:

- part1a/pcap1.pcap
- part1a/pcap2.pcap
- part1a/pcap3.pcap
- part1a/pcap4.pcap
- part1a/pcap5.pcap
- part1a/pcap6.pcap

1. List the different application layer protocols and their counts for each activity. In your report, specify how you figured out the protocol for each activity.
    - 1: ICMP (40)
    - 2: DNS (14), HTTPS (3208)
    - 3: DNS (6), HTTP (66), QUIC (373)
    - 4: HTTPS (8089), QUIC (2154)
    - 5: DNS (6), FTP (10)
    - 6: UDP (85)
2. How many HTTP and HTTPS packets did you record while performing activities 2 and 3?

2: 3208

3: 66

3. List the destination IP address used in each activity along with their timestamps. The destination IP address should be in the IPv4 format like x.x.x.x (e.g., “192.168.1.1”, “8.8.8.8”, “10.0.1.150”, etc.).

1: {'20.44.10.123', '13.66.138.105', '172.217.217.188', '140.82.116.6', '10.0.0.4'}

2: {'10.0.0.4', '172.217.217.188'}

3: {}

4: {'98.82.157.137', '104.18.26.193', '104.17.208.58', '151.101.43.52', '13.249.70.91', '23.206.229.111', '13.249.78.63', '10.0.0.4', '50.18.3.174', '52.45.101.96', '204.237.133.116', '44.244.162.28', '54.187.18.189', '34.211.131.132', '104.254.151.60', '18.173.121.78', '69.194.240.13', '151.101.0.134', '63.140.37.103', '44.239.221.55', '142.251.214.142', '54.187.74.22', '151.101.43.42', '13.249.74.121', '104.18.28.101', '3.169.183.5', '104.18.40.226', '35.186.253.211', '13.52.40.43', '52.33.195.54', '65.8.161.105'}

5: {'20.44.10.123', '13.66.138.105', '10.0.0.4', '17.248.192.1'}

6: {}

4. For activities 2, 3, and 4, can you tell which browser was used for these activities from the captured packets?

No, because 2 is HTTPS connection so you won’t be able to see headers, for 3 no http requests were being made so I could not see, and 4 is not done through browser.

Part 1b.

File locations:

- part1b/analyze.py

Secret Information: 'my-secret': 'Zubair Rocks!!'

Both PCAP1_2 and PCAP1_3 primarily capture local network activity involving Spotify Connect and AirPlay service discovery between nearby devices such as an iPhone and a Roku TV. Numerous mDNS packets advertise services like _spotify-connect._tcp.local, _airplay._tcp.local, and _apple-mobdev2._tcp.local, showing that devices were broadcasting their availability for media streaming and pairing on the same wireless network. In PCAP1_2, this is the main activity, suggesting someone was using or had the Spotify app open while the devices exchanged discovery messages. In PCAP1_3, that same background Spotify and AirPlay traffic is present, but the capture also includes DNS lookups for google.com, a short TLS handshake, and a sequence of ICMPv6 echo and time-exceeded messages. These indicate that, alongside the ongoing Spotify discovery, the user performed a traceroute or ping test to Google to check IPv6 connectivity.

Part 2.

File Locations:

- part2_udp/udp_serverAyaanPuri_920839614_VikramPenumarti_920928592.py
- part2_udp/udp_clientAyaanPuri__920839614_VikramPenumarti_920928592.py

In this part, we implemented a UDP client and server in Python to measure data transfer throughput. The client sent 100 MB of data to the server in 8 KB chunks (after reducing from 16 KB, which caused a “Message too long” error due to MTU limits). The server recorded the total bytes received and the time taken between the first and last packets, then calculated the

throughput and returned the result to the client. The server correctly received all 104,079,360 bytes and calculated a transfer time of about 0.051238 seconds, resulting in a measured throughput of approximately 1,983,667.85 KB/s. Because UDP does not guarantee delivery, the server occasionally received slightly fewer bytes than the total sent, which is expected behavior for a connectionless protocol.

Part 3.

File Locations:

- part3_proxy/proxy_serverAyaanPuri_920839614_VikramPenumarti_920928592.py
- part3_proxy/clientAyaanPuri__920839614_VikramPenumarti_920928592.py
- part3_proxy/serverAyaanPuri_920839614_VikramPenumarti_920928592.py
- part3_proxy/blocklistAyaanPuri__920839614_VikramPenumarti_920928592.txt

In this part, we implemented a TCP proxy in Python that acts as a middle man between a client and a destination server. The client sends a JSON object to the proxy containing the target server's IP, port, and message. The proxy reads this request, checks if the destination IP is blocklisted, and either rejects the request with an "Error" message or connects to the server and forwards the client's message. After receiving the server's reply, the proxy sends the response back to the client. For testing, a simple TCP server was used that responded with "pong" when receiving "ping". The client successfully communicated through the proxy and received the expected "pong" response. When a blocklisted IP was added, the proxy returned "Error", showing a correct blocking behavior.